FIAP

GLOBAL SOLUTION / 2026.2

CITIZEN TECH

RELATÓRIO TÉCNICO

Estruturas de Dados e Algoritmos

Citizen Assist AI | Sistema de serviços públicos



INTEGRANTES

Leonardo Scotti Tobias RM 573305

Natan Silva da Costa RM 573100

Henrique de Souza Aragão RM 570529

SÃO PAULO • 2026

RELATÓRIO TÉCNICO

CITIZEN TECH • DATA STRUCTURES AND ALGORITHMS

Global Solution 2026.2 | Curso: Ciência da Computação | FIAP

Resumo executivo

O Citizen Tech é um protótipo de gerenciamento e consulta de informações sobre serviços públicos. A solução organiza registros de RG, CPF, CNH e Assistência Social, com operações de cadastro, consulta, pesquisa, atualização, exclusão e análise. Seu núcleo utiliza uma lista simplesmente encadeada construída em Python, integrada a uma interface web por meio do Flask.

A ordenação dos serviços ocorre pelo identificador (ID), a partir de algoritmos implementados diretamente no backend. A versão consultada do repositório contém Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort e Tree Sort, além de registros de etapas e métricas para demonstração didática.

01 Contexto, objetivo e escopo

A proposta da Global Solution solicita a construção de um sistema que organize informações úteis ao cidadão, evidenciando o uso de estruturas de dados e algoritmos desenvolvidos pela equipe. O Citizen Tech adota como recorte os serviços de documentação e assistência social, representados por registros estruturados e acessíveis por uma interface web.

Objetivo geral

Implementar e demonstrar um sistema de gerenciamento de serviços públicos utilizando lista encadeada, busca sequencial, algoritmos de ordenação e operações completas de cadastro, leitura, atualização e remoção.

Objetivos específicos

- Cadastrar serviços com identificador numérico único gerado aleatoriamente.
- Permitir busca por ID, título e categoria, com retorno apropriado para consultas sem resultado.
- Demonstrar inserção, travessia e exclusão de nós da lista encadeada.
- Comparar algoritmos de ordenação pelo ID, registrando operações e etapas.
- Disponibilizar geração de registros fictícios e contagem de serviços por categoria.

Delimitação

O sistema não emite documentos nem consulta cadastros pessoais oficiais. Os dados de demonstração são fictícios; a aplicação não depende de APIs governamentais, autenticação de usuários ou banco de dados. A persistência em arquivo não faz parte desta versão.

02 Requisitos funcionais e modelagem

Operação	Comportamento implementado
Cadastrar	Recebe título, categoria, descrição, requisitos e local; cria um ID aleatório não utilizado.
Consultar	Localiza um serviço pelo ID informado.
Pesquisar	Busca por título ou categoria, sem diferenciar maiúsculas de minúsculas.

Operação	Comportamento implementado
Listar	Percorre os nós e retorna os registros armazenados.
Atualizar	Modifica os campos do registro identificado, preservando seu ID.
Excluir	Remove o nó da lista, reajustando as referências entre os nós.
Analisar	Informa total de registros e contagem por categoria.
Ordenar	Executa um dos seis algoritmos e devolve sequência, métricas e passos.
Gerar	Insere cinco registros fictícios para exercitar as operações.

Modelo de serviço

Atributo	Tipo	Finalidade
id	int	Identificador único no conjunto ativo de registros; faixa de 1000 a 9999.
titulo	str	Nome apresentado na listagem e pesquisável por texto.
categoria	str	RG, CPF, CNH ou Assistência Social.
descricao	str	Descrição do serviço.
requisitos	str	Requisitos informativos para o atendimento.
local	str	Local de atendimento registrado.

O modelo é representado pela classe Servico, definida em backend/models/servico.py. O nó da lista é outra classe, No, com os atributos servico e proximo.

03 Arquitetura e organização do projeto



Figura 1 — Fluxo entre interface, backend e estrutura de dados (elaboração para esta documentação).

A interface HTML/CSS/JavaScript envia solicitações HTTP para o backend Flask. A aplicação invoca as funções de gerenciamento, que acessam os algoritmos de busca e ordenação e operam sobre os registros mantidos em memória pela lista encadeada.

```
GS_DSA/
  backend/
    algoritmos/   busca.py  |  ordenacao.py
    models/       servico.py  |  no.py
    services/     gerenciamento.py
    structures/   lista.py  |  arvore.py
    main.py       teste.py
  frontend/       index.html  |  style.css  |  script.js
  requirements.txt
```

Responsabilidades dos módulos

Módulo	Responsabilidade

Módulo	Responsabilidade
models	Representação do serviço e do nó encadeado.
structures	ListaEncadeada e NoArvore para Tree Sort.
algoritmos	Busca sequencial e seis métodos de ordenação.
services	Regras de negócio e validação dos registros.
main.py	Rotas Flask, tratamento de requisições e serialização JSON.
frontend	Formulários, botões, resultados e passos visuais.
teste.py	Casos automatizados para ordenação e operações básicas da lista.

04 Lista encadeada e operações CRUD



Figura 2 — Encadeamento lógico de registros; IDs meramente ilustrativos.

A classe ListaEncadeada utiliza as referências inicio e ultimo. Um novo No contém o objeto Servico e o atributo proximo, inicialmente None. Na inserção ao final, o nó anterior passa a referenciar o novo nó, enquanto ultimo é atualizado. Isso evita procurar o final a cada cadastro.

Inserção e percurso

A inserção é feita no fim da lista. O percurso começa em inicio, acompanha proximo e termina ao encontrar None. A listagem devolve uma lista auxiliar de objetos Servico, construída a partir dos nós encadeados.

Remoção e atualização

A remoção percorre a estrutura com referências atual e anterior. Ao encontrar o ID, reconecta os nós vizinhos; a exclusão do primeiro elemento atualiza inicio, e a exclusão do último atualiza ultimo. A atualização localiza o objeto pelo ID e altera os campos do serviço, sem substituir sua posição na lista.

IDs aleatórios e integridade

Os IDs são sorteados no intervalo [1000, 9999] e verificados por busca sequencial antes da inserção. Caso haja colisão com um registro ativo, um novo ID é sorteado. A geração do ID, portanto, não equivale a um índice de posição; a lista pode conter IDs em qualquer ordem.

05 Busca e métodos de ordenação

Busca sequencial

A função buscar_id visita cada nó até encontrar igualdade entre o ID consultado e o ID do registro. Para título e categoria, buscar_texto percorre toda a lista e utiliza correspondência parcial, sem diferenciar letras maiúsculas de minúsculas (casefold). A consulta por ID retorna ausência quando não encontra o registro; a camada de gerenciamento converte esse caso em mensagem de erro para a interface.

Ordenação dos serviços por ID

Todos os algoritmos recebem uma cópia dos registros retornados pela listagem, de modo que a ordem da lista encadeada original não é modificada. O método executar(nome, registros) retorna a sequência ordenada, um dicionário de métricas e o histórico das etapas.

Algoritmo	Procedimento adotado	Métrica em destaque
Bubble Sort	Compara pares vizinhos e troca elementos fora de ordem.	Comparações e trocas.
Selection Sort	Localiza o menor ID na parte ainda não ordenada.	Comparações e trocas.
Insertion Sort	Desloca IDs até posicionar a chave no trecho ordenado.	Comparações e movimentações.
Merge Sort	Divide a sequência e intercala os blocos ordenados.	Comparações e movimentações.
Quick Sort	Usa o último ID da partição como pivô e particiona recursivamente.	Comparações e trocas.
Tree Sort	Insere em árvore binária de busca e realiza percurso em ordem.	Comparações, inserções e movimentações.

Visualização passo a passo

Cada algoritmo registra estados intermediários com texto explicativo, sequência de IDs e instantâneo das métricas acumuladas. A interface permite avançar ou retornar etapas. Em Merge Sort, alguns passos representam sub-blocos; no Tree Sort, a sequência textual representa o percurso e as inserções, mas não um desenho completo da topologia da árvore.

06 Análise de complexidade Big-O

Considere n como o número de serviços ativos. As análises abaixo descrevem o trabalho principal dos algoritmos e separam o custo do registro de etapas utilizado pela interface.

Operação	Melhor caso	Pior caso	Justificativa
Inserção no fim	O(1)	O(1)	A lista mantém referência ultimo.
Percurso/listagem	O(n)	O(n)	Visita todos os n nós.
Busca por ID	O(1)	O(n)	Pode encontrar no primeiro nó ou percorrer tudo.
Pesquisa textual	O(n)	O(n)	Verifica cada registro; desconsidera custo do tamanho do texto.
Remoção por ID	O(1)	O(n)	Ajuste de ponteiros é constante, busca é linear.
Atualização por ID	O(1)	O(n)	Depende da localização prévia do nó.
Contagem por categoria	O(n)	O(n)	Percorre integralmente os registros.

Quadro 1 — Custos de lista encadeada e busca, desconsiderando serialização e visualização.

Algoritmo	Melhor caso	Médio	Pior caso
Bubble Sort com parada antecipada	O(n)	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$
Merge Sort	O(n log n)	O(n log n)	O(n log n)
Quick Sort (último pivô)	O(n log n)	O(n log n)	$O(n^2)$

Algoritmo	Melhor caso	Médio	Pior caso
Tree Sort (BST não balanceada)	O(n log n)	O(n log n)*	$O(n^2)$

* Para Tree Sort, o caso médio assume uma ordem de inserção aleatória, em que a árvore tende a apresentar altura logarítmica. Uma sequência de IDs já ordenada pode degenerar a árvore em cadeia, elevando o custo a $O(n^2)$.

Distinção importante: a inserção estrutural no fim da lista é O(1), mas o cadastro completo também verifica se o ID sorteado já existe, percorrendo até n nós por tentativa. Portanto, o custo esperado inclui ao menos uma busca O(n), podendo haver novas tentativas quando ocorrer colisão de IDs.

Custo adicional da implementação: antes de ordenar, a aplicação converte a lista encadeada para uma lista Python, operação O(n) em tempo e espaço. O histórico de passos replica listas de IDs em cada registro; em alguns algoritmos, essa instrumentação aumenta significativamente memória e tempo reais, sem alterar a lógica teórica das operações centrais apresentada no quadro.

07 Interface web e fluxo de operação

A aplicação expõe a página principal por GET / e recursos estáticos por GET /assets/<nome>. A rota POST /acao/<acao> recebe JSON e delega cada ação a uma função do módulo gerenciamento. As respostas retornam registros, análise ou resultados de ordenação em JSON; erros de entrada tratados retornam HTTP 400.

Ação na interface	Operação no backend	Saída esperada
Cadastrar	cadastrar_servico	Registro inserido com ID aleatório.
Consultar ID	pesquisar_id	Um registro ou erro de ID inexistente.
Buscar título / categoria	buscar_titulo / buscar_categoria	Lista de correspondências.
Listar	mostrar_lista	Registros em ordem de encadeamento.
Atualizar / excluir	atualizar / excluir	Atualização do conjunto de registros.
Gerar 5 fictícios	gerar	Inserção de dados simulados.
Analisar	analisar	Total e contagem por categoria.
Ordenar	ordenar	Registros ordenados, métricas e passos.
Limpar	Atribuição de inicio e ultimo a None	Estrutura esvaziada em memória.

Exemplo didático de ordenação

```
Entrada: [50, 20, 80, 10, 30]
Critério: ID crescente
Resultado esperado: [10, 20, 30, 50, 80]
```

Durante a demonstração, o usuário escolhe um método de ordenação e acompanha a progressão dos IDs e dos contadores. Os resultados são exibidos em uma cópia; as consultas futuras continuam utilizando o encadeamento originalmente cadastrado.

08 Testes e critérios de validação

O arquivo backend/teste.py contém oito conjuntos de entradas e executa seis métodos de ordenação sobre cada um, totalizando 48 verificações por assert de equivalência com a ordem esperada. A função sorted() é usada apenas como referência nos testes, e não como mecanismo de ordenação do sistema.

Caso de teste	Entrada	Resultado esperado
Não ordenada	[50, 20, 80, 10, 30]	[10, 20, 30, 50, 80]
Lista vazia	[]	[]
Elemento único	[1]	[1]
Já ordenada	[1, 2, 3]	[1, 2, 3]

Caso de teste	Entrada	Resultado esperado
Ordem inversa	[3, 2, 1]	[1, 2, 3]
Nove IDs	[9, 4, 7, 2, 8, 1, 6, 3, 5]	[1, 2, 3, 4, 5, 6, 7, 8, 9]
Valores diversos	[100, 500, 300, 200, 400]	[100, 200, 300, 400, 500]
IDs repetidos em teste isolado	[5, 5, 2, 2, 1]	[1, 2, 2, 5, 5]

O mesmo arquivo verifica operações de busca por ID existente e inexistente, remoção do primeiro/último nó, esvaziamento da lista e tentativa de remoção em estrutura vazia. Esses testes examinam a lógica básica das estruturas.

Casos adicionais recomendados

- Cadastrar um serviço com campo em branco e verificar a mensagem de validação.
- Solicitar atualização e exclusão com ID inexistente.
- Pesquisar por título ou categoria sem correspondências.
- Executar todas as ações com lista vazia pela interface.
- Simular IDs gerados em colisão e confirmar que não são duplicados entre os serviços ativos.
- Executar o fluxo completo: cadastrar → consultar → atualizar → pesquisar → excluir → consultar novamente.

Estado da evidência: esta documentação identifica os casos existentes no código e os resultados esperados. Não foram anexados logs, capturas de tela ou saída de execução da versão consultada; a realização e o registro desses testes devem ser demonstrados pela equipe na entrega.

09 Instalação e demonstração prática

Requisitos

Python instalado, gerenciador pip e navegador web atualizado. A dependência principal indicada em requirements.txt é Flask.

```
python -m pip install -r requirements.txt
python backend/teste.py
python backend/main.py
```

Com o servidor iniciado, abrir http://127.0.0.1:5000 no navegador. Não basta abrir frontend/index.html diretamente: a página depende do backend Flask para executar as ações.

Roteiro recomendado de apresentação

- Iniciar com a lista vazia e demonstrar a mensagem de ausência de registros.
- Cadastrar manualmente um serviço; destacar a geração automática de ID.
- Gerar cinco registros fictícios e exibir a lista encadeada em ordem de inserção.
- Consultar por ID, pesquisar título e categoria; demonstrar tratamento de resultados inexistentes.
- Atualizar e excluir um registro; consultar novamente para confirmar a alteração.
- Executar Merge Sort e Quick Sort; navegar pelas etapas e comparar métricas.
- Executar teste.py no terminal e apresentar o resultado real dos asserts.

10 Limitações, melhorias e conclusão

Limitações observadas no código

- Os registros são mantidos em memória; ao reiniciar a aplicação, o conjunto cadastrado se perde.
- Os dados fictícios não devem ser confundidos com orientações governamentais oficiais.
- A geração aleatória de IDs exige busca para evitar colisões e limita-se à faixa de 1000 a 9999.
- A interface apresenta ações de gerenciamento, mas não contém botão explícito de encerramento.
- Os testes automatizados publicados concentram-se em ordenação e lista; há espaço para maior cobertura do CRUD e de entradas inválidas.
- O número de passos visuais cresce com o volume de dados; para conjuntos maiores, recomenda-se limitar a instrumentação.

Conclusão

O Citizen Tech materializa os principais conceitos de Data Structures and Algorithms por meio de uma aplicação de serviços públicos. O uso explícito de nós e referências permite compreender inserção, travessia, atualização e remoção; a busca sequencial evidencia o custo linear de consultas; e os seis métodos de ordenação oferecem base concreta para observar diferentes estratégias e complexidades. A interface web funciona como camada de demonstração, preservando as implementações centrais em Python.

Para a finalização acadêmica, recomenda-se complementar o relatório com evidências reais de execução, revisar as entradas inválidas e registrar evidências dos testes realizados.